

Uke 6

Opprette skjema

- `\dn` – lister opp skjemaer på databasen
- `create schema uke6;` – lager et nytt skjema med navnet uke6
- `\dt uke6.;` – lister opp alle tabeller i skjemaet
- `\dv uke6.;` – lister opp alle views i skjemaet
- `set schema 'uke6';` – endrer til uke6 skjemaet
- `drop schema uke6 cascade;` – sletter skjema

Datatype

- int
- bigint (store heltall)
- text – ubegrenset lengde
- varchar(n) – samme som tekst bare med maks lengde
- char – en bokstav

Merk, Bruk varchar(n) eller char(n) kun som skranker.

- real – flyttall
- boolean
- date – 'yyyy-mm-dd'
- time – hh:mm', time 'hh:mm:ss'
- timestamp – kombinasjon av dato og tid: 'yyyy-mm-dd hh:mm:ss'
- serial – heltall serie 1,2,3...
- numeric(n,d) – n totale siffer, d siffer etter komma

Numeriske typer

- Ceil/floor – runder opp/ned
- Round – runder av til nærmeste heltall
- Sqrt, power – kvadratroten og potens-funksjon
- Abs – absoluttverdi
- Sin, cos, tan
- Random
- Se følgende for alle: <https://www.postgresql.org/docs/11/functions-math.html>

Create table

På formen:

```
CREATE TABLE tabellnavn ( <kolonne1>, <kolonne2> ... );
```

Eksempel:

```
CREATE TABLE Person (  
    Fodselsnummer int PRIMARY KEY,  
    Navn text NOT NULL,  
    Fdag date  
);
```

Dette vil gi tabellen:

```
Person(Fodselsnummer, Navn, [Fdag])  
PN: Fodselsnummer
```

- Merk, kan kun ha én **PRIMARY KEY** per tabell, må bruke **UNIQUE NOT NULL** dersom vi har flere kandidatnøkler.
- For primærnøkler med heltall kan man bruke **SERIAL PRIMARY KEY**

Create table

- Hvis en PN består av flere kolonner, må man bruke **CONSTRAINT**, slik som dette:

Eksempel:

```
CREATE TABLE Person (  
    Personnummer int NOT NULL,  
    Navn text NOT NULL,  
    Fdag int NOT NULL,  
    CONSTRAINT pn PRIMARY KEY (Fdag, Personnummer)  
);
```

Merk, det gjelder også kandidatnøkler.

Referere til andre tabeller

La oss si vi har tabellen:

Timeliste(timelistenr, status, levert, utbetalt, beskrivelse)

```
CREATE TABLE Timelistelinje (  
    timelistenr int REFERENCES Timeliste(timelistenr),  
    linjenr int,  
    startdato date NOT NULL,  
    starttid time NOT NULL,  
    sluttid time,  
    pause int,  
    beskrivelse text NOT NULL,  
    PRIMARY KEY (timelistenr, linjenr)  
);  
  
FOREIGN KEY (timelistenr) REFERENCES Timeliste(timelistenr) *ALTERNATIVT*
```

Merk, Den alternative måten må brukes om det er flere kolonner som refererer til en annen tabell.

CHECK – CONSTRAINT

- Brukes når man oppretter en tabell.
- Er en begrensning som lar deg angi om verdier i en kolonne må oppfylle bestemte krav.
- Hindrer folk å inserte eller update ugyldige verdier i en kolonne/tabell

Eks:

```
CREATE TABLE Timeliste (  
    timelistenr int PRIMARY KEY,  
    status text NOT NULL,  
    CHECK (status = 'aktiv' OR status = 'levert' OR status = 'utbetalt')  
);
```

I denne tabellen kan man kun inserte eller update status som 'aktiv', 'levert' eller 'utbetalt'.

Prøver man f.eks å inserte status = 'sent', så vil man få en error

Insert into

```
INSERT INTO tabell  
VALUES ( <rad1> ),  
        ( <rad2> ),  
        ( <rad3> );
```

```
INSERT INTO Timeliste  
VALUES  
(0, 'aktiv'),  
(1, 'aktiv'),  
(2, 'levert');
```

INSERT INTO

- Inserte med en spørring som henter data fra andre tabeller

```
CREATE TABLE Students2018 (  
    SID int PRIMARY KEY,  
    StdName text NOT NULL  
);  
  
INSERT INTO Students2018  
SELECT S.SID , S.StdName  
FROM Students AS S INNER JOIN TakesCourse AS T ON (S.SID = T.SID)  
WHERE T.Semester LIKE '%18 ';
```

INSERT INTO

- Kan inserte rett i tabellen mens man oppretter den, det gjøres slik:

```
CREATE TABLE Students2018 AS  
SELECT S.SID , S.StdName  
FROM Students AS S INNER JOIN TakesCourse AS T ON (S.SID = T.SID)  
WHERE T.Semester LIKE '%18 ';
```

- Dette gir samme data, men merk at vi nå ikke har skrankene PRIMARY KEY og NOT NULL
- Disse må da legges til etterpå

Oppdatere ting

- Endre navn på tabeller:

```
ALTER TABLE Students
```

```
RENAME TO UIOStudents ;
```

- Legge til en kolonne:

```
ALTER TABLE Courses
```

```
ADD COLUMN Teacher text;
```

- Vi kan også legge til skranker etter at en tabell er laget.

```
ALTER TABLE Courses
```

```
ADD CONSTRAINT cid_pk PRIMARY KEY (cid );
```

Slette ting

- For å slette rader fra en tabell bruker vi delete:

```
DELETE  
FROM <tabellnavn >  
WHERE <betingelse >
```

Eks: Sletting av alle studenter født etter 1990-01-01 gjøres slik:

```
DELETE  
FROM Students  
WHERE StdBirthdate > '1990 -01 -01 '
```

Oppdatere data

- ◆ **UPDATE** lar oss oppdatere verdiene i en tabell:

```
UPDATE <tabellnavn>  
    SET <oppdateringer>  
    WHERE <betingelse>
```

hvor <oppdateringer> er en liste med oppdateringer som blir eksekvert for hver rad som gjør <betingelse> sann

- ◆ For eksempel:

```
UPDATE Students  
    SET StdBirthdate = '1987-10-03'  
    WHERE StdName = 'Sam Penny'
```

oppdaterer fødselsdatoen til studenten Sam Penny til '1987-10-03'

- ◆ Mens

```
UPDATE products  
    SET unit_price = unit_price * 1.1  
    WHERE quantity_per_unit LIKE '%bottles%'
```

øker prisen med 10% på alle produkter som selges i flasker i products-tabellen

VIEWS

- Vi bruker spørringer for å få ut interessant data
- Av og til vil en bestemt spørring bli eksekvert veldig ofte. Det er da upraktisk å måtte skrive den ut hver gang
- Et view kan så brukes som om det var en vanlig tabell
- Men blir beregnet på nytt hver gang den brukes
- Så et view tar ikke opp noe plass og trengs ikke oppdateres

Views og With

```
CREATE VIEW NAVN AS (  
  SELECT ...  
  FROM ...  
  WHERE ...  
)
```

```
WITH NAVN AS (  
  SELECT ...  
  FROM ...  
  WHERE ...  
)
```

Forskjellen på VIEWS og WITH:

- **VIEWS** lager et faktisk objekt i databasen
- **WITH** er bare en del av en enkelt spørring